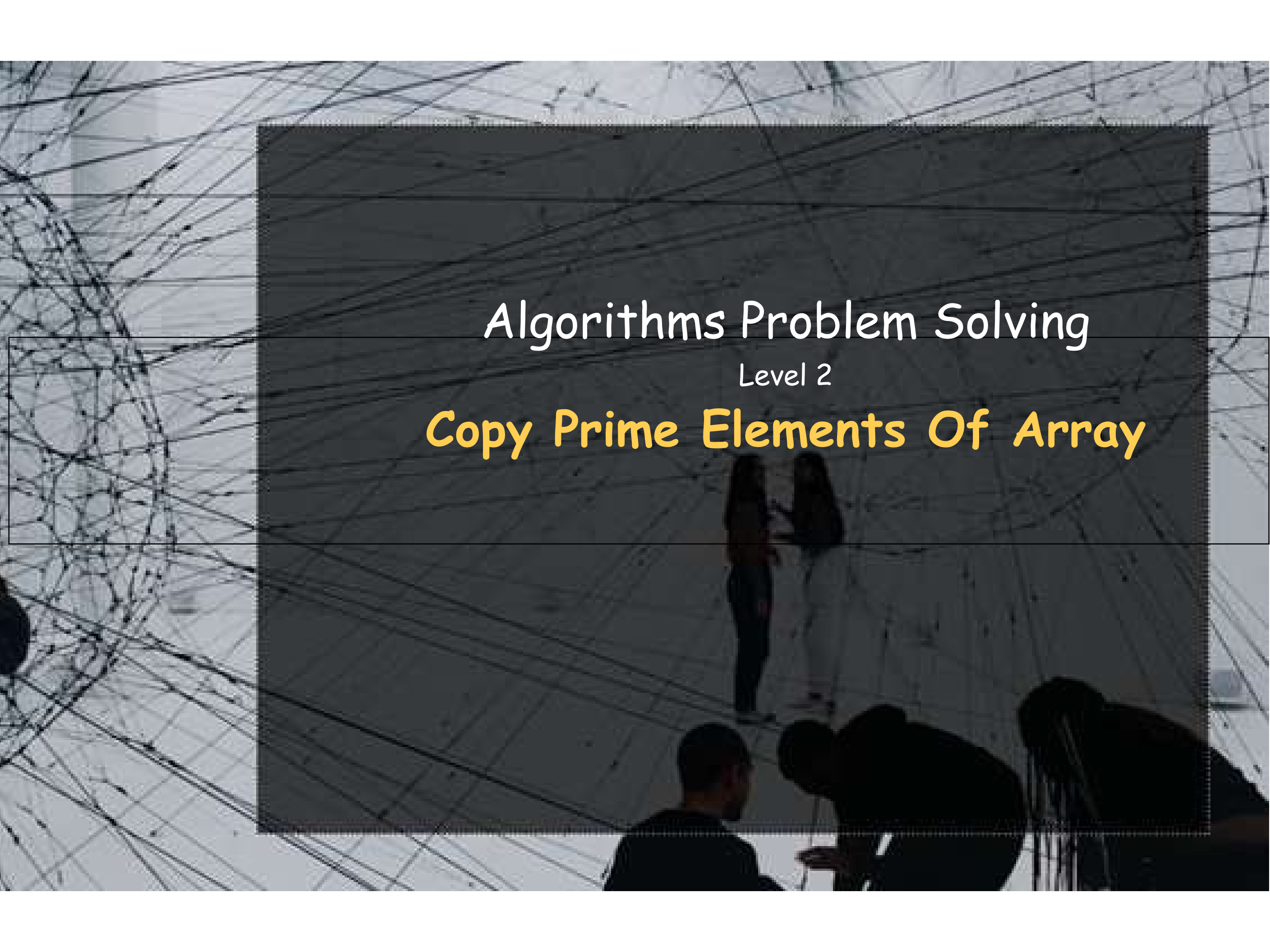


Algorithms Problem Solving Level 2

Copy Prime Elements Of Array



Algorithms Problem Solving

Level 2

Copy Prime Elements Of Array

Definition

- ◆ This program fills an array with random numbers from `1` to `100`.
- ◆ It checks each element to determine whether it is a prime number.
- ◆ Only the prime numbers are copied from the first array into a second array.
- ◆ Finally, the second array containing only prime numbers is printed.

Explanation

- ◆ ``IsPrime()`` checks whether a number is prime.
- ◆ ``CopyPrimeNumbersInArray()`` loops through ``arr1`` and copies only the elements for which ``IsPrime()`` returns ``Prime``.
- ◆ ``counter`` keeps track of how many prime numbers have been copied.
- ◆ When a prime number is found:
 `arr2[counter] = arr1[i];`
 `counter++;`
- ◆ This means the prime number is stored in the next available position in ``arr2``.
- ◆ The enum:
 `enum enPrime { Prime = 1, NotPrime = 2 };`
- ◆ is used to make the result of ``IsPrime()`` easier to understand.

Steps in main()

1. Fill `arr1` with random numbers from `1` to `100`.
2. Loop through every element in `arr1`.
3. Call `IsPrime()` for each element.
4. If the number is prime, copy it to `arr2`.
5. Increase `counter` after each copied prime number.
6. Store the number of copied elements in `arr2Length`.
7. Print the original array.
8. Print the array containing only prime numbers.

💡 Example

◆ `Array 1`:

41 89 90 86 6 54 96 84 95 100

◆ Check each number:

41 → Prime

89 → Prime

90 → Not Prime

86 → Not Prime

6 → Not Prime

54 → Not Prime

96 → Not Prime

84 → Not Prime

95 → Not Prime

100 → Not Prime

◆ Therefore, `arr2` becomes:

41 89

➡ Result

Array 1 Elements: 41 89 90 86 6 54 96 84 95 100

Array 2 Elements (Primes): 41 89

◆ The example in the original comment shows only `41`, but `89` is also a prime number, so the correct result for that exact array is `41 89`.

Note

- ◆ The `IsPrime()` function uses:
 `for (int i = 2; i * i <= num; i++)`
- ◆ This checks possible divisors only up to the square root of the number, which is more efficient than checking all numbers up to `num / 2`.
- ◆ `counter` starts at `0` because array indexes start at `0`.
- ◆ `arr2Length` should contain the number of prime elements copied.
- ◆ The correct final statement should be:
 `arr2Length = counter;`

◆ There is a bug in the given code:

```
arr2Length = --counter;
```

◆ `--counter` decreases the counter by `1`, so the reported array length becomes incorrect.

◆ For example, if `2` prime numbers are copied, `counter` becomes `2`, but `--counter` changes it to `1`.

◆ This causes the program to print only one prime instead of two.

◆ If there are no prime numbers, `counter` remains `0`, and `--counter` makes `arr2Length` equal to `-1`, which is invalid.

◆ It should be:

```
arr2Length = counter;
```

◆ The array length should also be validated so `arr1Length` does not exceed `100`.

Comparison

`IsPrime()`

- ✓ Checks whether one number is prime.
- ✓ Returns `Prime` or `NotPrime`.
- ✓ Example: `41` → `Prime`.

`CopyPrimeNumbersInArray()`

- ✓ Checks all elements of the array.
- ✓ Copies only the elements identified as prime.
- ✓ Example: `41 90 89 100` → `41 89`.

✦ Summary

- ✓ Fill the first array with random numbers.
- ✓ Check every element using `IsPrime()`.
- ✓ Copy only prime numbers into the second array.
- ✓ Use `counter` to track the position in `arr2`.
- ✓ `arr2Length` must be set to `counter`.
- ✓ Do not use `--counter`, because it decreases the correct number of copied elements.

Thank you 😊